

Activation Functions — Complete Cheatsheet with Code

This cheatsheet summarizes the most common activation functions used in machine learning and deep learning, including their mathematical formulas, characteristics, and implementation code in NumPy and PyTorch. Content inspired by the Kaggle discussion 'Activation Functions — Cheatsheet / Complete Overview with Code'.

Sigmoid

Formula: $\sigma(x) = 1 / (1 + e^{(-x)})$

Notes:

- Range: (0, 1)
- Smooth gradient, but can cause vanishing gradients for large $|x|$
- Used in logistic regression and output layers for probabilities

Code Example:

```
# NumPy import numpy as np def sigmoid(x): return 1 / (1 + np.exp(-x)) # PyTorch import torch torch.sigmoid(torch.tensor(x))
```

Tanh

Formula: $\tanh(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$

Notes:

- Range: (-1, 1)
- Zero-centered; stronger gradients than sigmoid
- Still suffers from saturation at extreme values

Code Example:

```
# NumPy def tanh(x): return np.tanh(x) # PyTorch torch.tanh(torch.tensor(x))
```

ReLU

Formula: $f(x) = \max(0, x)$

Notes:

- Range: $[0, \infty)$
- Simple and efficient; avoids saturation for positive inputs
- Can cause 'dead neurons' when $x < 0$

Code Example:

```
# NumPy def relu(x): return np.maximum(0, x) # PyTorch import torch.nn.functional as F F.relu(torch.tensor(x))
```

Leaky ReLU

Formula: $f(x) = x$ if $x > 0$ else αx

Notes:

- Fixes dead ReLU issue by allowing small slope α for $x < 0$
- Typical $\alpha = 0.01$

Code Example:

```
# NumPy def leaky_relu(x, alpha=0.01): return np.where(x > 0, x, alpha * x) # PyTorch
F.leaky_relu(torch.tensor(x), negative_slope=0.01)
```

ELU

Formula: $f(x) = x$ if $x > 0$ else $\alpha(e^x - 1)$

Notes:

- Smooth for $x < 0$; helps learning in noisy environments
- Hyperparameter α controls the curve for negatives

Code Example:

```
# NumPy def elu(x, alpha=1.0): return np.where(x > 0, x, alpha * (np.exp(x) - 1)) #
PyTorch F.elu(torch.tensor(x), alpha=1.0)
```

Softmax

Formula: $\sigma(x_i) = e^{x_i} / \sum_j e^{x_j}$

Notes:

- Converts logits to probabilities
- Used in multi-class classification output layers

Code Example:

```
# NumPy def softmax(x): e_x = np.exp(x - np.max(x)) return e_x / e_x.sum(axis=0) #
PyTorch F.softmax(torch.tensor(x), dim=0)
```